PI AGENT

Slash Commands

A companion study guide

> A person can outsource intelligence,
> but not understanding.

The complete fixed command set, the conditional command, dynamic commands, and the events that commands can cause.

Designed to accompany:
Pi Agent - Hooks, Events, Commands and Handlers

Prepared 24 July 2026

1. The short mental model

A slash command is a direct instruction to the Pi harness. Pi normally handles it itself instead of sending it to the language model.

You type /command -> **Pi performs an action** -> **Events may occur**

Command = what you ask Pi to do.
Action = what Pi changes or opens.
Event = a named moment Pi announces while that work happens.
Hook = your listener, usually written with `pi.on(...)`.

```
/compact is therefore a command. Compaction is the action. session_before_compact and
session_compact are events. Your pi.on() registrations are hooks.
```

How many slash commands are there?

22 fixed core commands are defined in Pi's built-in command source. The official README also documents /llama, a shipped but conditional command for the llama.cpp router. That gives 23 shipped commands to learn.

The final total on your screen can be larger. Skills, prompt templates and extensions can add commands dynamically, so there is no permanent maximum.

Three useful categories

`22 core`	Always part of Pi's fixed built-in command list.
`+ /llama`	Available when using the llama.cpp router; manages local GGUF models.
`+ dynamic`	Loaded skills, prompt templates and extensions can each add commands.

2. Complete quick reference - part 1

The event column names only events that Pi extensions can reliably observe. "None guaranteed" does not mean nothing happens; it means there is no dedicated extension lifecycle event promised for that command.

Command	Plain-language purpose	Related extension events
`/login [provider]`	Sign in to an AI provider.	None guaranteed.
`/logout`	Remove stored provider credentials.	None guaranteed.
`/model [provider/model]`	Choose the model used for later turns.	model_select; sometimes thinking_level_select.
`/scoped-models`	Choose which models Ctrl+P cycles through.	None guaranteed.
`/settings`	Open Pi's settings screen.	May emit thinking_level_select when that value changes.
`/resume`	Switch to a saved session.	Switch lifecycle.
`/new`	Start a fresh session.	Switch lifecycle.
`/name <name>`	Give the current session a useful name.	session_info_changed.
`/session`	Show current session facts and usage.	None guaranteed.
`/tree`	Move to another point in this session's history.	Tree lifecycle.
`/trust`	Save the trust choice for this project.	None guaranteed.

The conditional shipped command

Command	Plain-language purpose	Related extension events
`/llama`	Open the llama.cpp router manager: load, unload or download local GGUF models.	None guaranteed.

Remember: only models already loaded through the router appear in `/model`. If the router is not part of your setup, `/llama` may not be available.

3. Complete quick reference - part 2

Command	Plain-language purpose	Related extension events
`/fork`	Branch from an earlier user message into a new session.	Fork lifecycle.
`/clone`	Copy the current branch into a new session.	Fork lifecycle.
`/compact [instructions]`	Summarize older context so the active conversation is smaller.	Compact lifecycle.
`/copy`	Copy the last assistant message to the clipboard.	None guaranteed.
`/export [file]`	Export the session; HTML by default, JSONL with a .jsonl filename.	None guaranteed.
`/import <file>`	Replace the current session with an imported JSONL session.	Switch lifecycle.
`/share`	Upload a secret GitHub gist containing shareable HTML.	None guaranteed.
`/reload`	Reload extensions, skills, prompts and themes.	Reload lifecycle.
`/hotkeys`	Show keyboard shortcuts.	None guaranteed.
`/changelog`	Show Pi's recent changes.	None guaranteed.
`/quit`	Close Pi.	session_shutdown.

Not slash commands

`!command` runs a shell command and includes its output in model context. `!!command` runs it without including the output. These are shell prefixes, not slash commands. They are related to the `user_bash` event.

4. Providers, models and session basics

Providers and models

`/login [provider]`	Connect an AI provider. Supplying the provider name can take you directly to it.
`/logout`	Remove the selected provider's saved authentication.
`/model [provider/model]`	Choose the active model. With no argument, Pi opens the model selector.
`/scoped-models`	Limit the models available in the quick Ctrl+P cycle. This does not delete models.
`/settings`	Open preferences such as thinking level, theme, message delivery and transport.
`/llama`	When the llama.cpp router is configured, load or unload a local GGUF model, or download one from Hugging Face.

Session basics

`/new`	Begin a new, empty session.
`/resume`	Choose a saved session and make it current.
`/name <name>`	Name the current session so it is easier to recognize later.
`/session`	Inspect the current file, session ID, message count, tokens and cost.
`/trust`	Store your future trust decision for the current project. Pi must be restarted for the new decision to take effect.

Useful distinction: `/new` creates a new session; `/resume` switches to one that already exists.

5. History, branching and context

These four commands are easy to confuse because each changes what you are working with, but they do not do the same thing.

`/tree`	Move inside the same session. Choose another point or branch in its history. The history remains in one session file.
`/fork`	Create a new session from an earlier user message. Pi places that selected prompt in the editor so you can revise it.
`/clone`	Create a new session from the current branch at its present point. The editor starts empty.
`/compact [instructions]`	Reduce active context. Pi summarizes older messages and keeps recent material. Optional instructions tell it what the summary must preserve.

One-line decision guide

Command	Plain-language purpose	Related extension events
`/tree`	I want to revisit another point in this same session.	Tree lifecycle.
`/fork`	I want a new path beginning from an older prompt.	Fork lifecycle.
`/clone`	I want a separate copy of where I am now.	Fork lifecycle.
`/compact`	I want to continue here with a smaller active context.	Compact lifecycle.

Compaction is intentionally lossy for active context. The complete history still remains in the JSONL session file; the model receives the summary plus the recent messages.

```
Example: /compact Preserve our API decisions, unresolved bugs, filenames and next steps.
```

6. Files, sharing, runtime and help

Copy, export, import and share

`/copy`	Copy the most recent assistant message to your clipboard.
`/export [file]`	Save the session as HTML by default. Use a filename ending in .jsonl for a JSONL export.
`/import <file>`	Import a JSONL session and replace the current session after confirmation.
`/share`	Create a secret/unlisted GitHub gist containing shareable HTML. The GitHub CLI must be installed and authenticated.

Export versus import: export writes a copy; import changes which session is current. Treat import as a session switch.

Runtime and help

`/reload`	Reload extensions, skills, prompt templates and themes without leaving Pi.
`/hotkeys`	Display the keyboard-shortcut reference.
`/changelog`	Display recent changes to Pi.
`/quit`	Close Pi cleanly.

After `/reload`, newly added resources can become available. If an extension registers a command, that command may then appear in your slash-command suggestions.

7. Which commands cause which events?

This is the bridge back to the hooks-and-events guide. A command is the cause; these events are observable moments in the resulting lifecycle.

Command	Typical event chain
`/new or /resume`	session_before_switch -> session_shutdown -> session_start -> resources_discover
`/fork or /clone`	session_before_fork -> session_shutdown -> session_start -> resources_discover
`/name`	session_info_changed
`/tree`	session_before_tree -> session_tree
`/compact`	session_before_compact -> session_compact
`/import`	session_before_switch -> session_shutdown -> session_start; resource discovery follows runtime initialization
`/model`	model_select; thinking_level_select may occur if Pi clamps the thinking level
`/settings`	thinking_level_select only when the thinking level is changed there
`/reload`	session_shutdown(reason: reload) -> session_start(reason: reload) -> resources_discover(reason: reload)
`/quit`	session_shutdown(reason: quit)

A hook listening to compaction therefore looks like:

```
pi.on("session_before_compact", async (event, ctx) => {
  // Run when compaction is about to begin.
});
```

Important: the command and hook are not alternatives. `/compact` asks for the work; `pi.on(...)` listens for a moment in that work.

8. Commands added dynamically

The fixed list is complete, but the list visible in a particular Pi installation can grow in three ways.

`/skill:<name>`	A command created for each loaded skill. Example: /skill:review.
`/<template-name>`	A command created from a prompt-template file. The template's name becomes the command.
`Extension command`	An extension can call pi.registerCommand(...) and choose its own slash-command name and behavior.

Because extensions choose their own names, nobody can publish one finite list of every possible slash command. The 22 core + conditional /llama list in this guide is the stable shipped set; the rest depends on what you load.

A final memory aid

Slash command: "Pi, do this."
Event: "This moment has happened."
Hook: "When that moment happens, run my handler."

Official sources

- Pi coding-agent README - interactive commands and sessions
- Built-in slash-command source - the 22 fixed core commands
- Extension documentation - lifecycle events and custom commands
- llama.cpp router documentation - the conditional /llama command
- Session runtime source - import and switching lifecycle

Version note: checked against the official Pi repository on 24 July 2026. Commands can change as Pi evolves; dynamic commands depend on your local resources.